Release Notes

Java Card™ 2.1.1 Specifications



Sun Microsystems, Inc.
901 San Antonio Road
Palo Alto, CA 94303 USA
650 960-1300 fax 650 969-9131

Revision 1.0
May 18, 2000

Copyright © 2000 Sun Microsystems, Inc.

901 San Antonio Road, Palo Alto, CA 94303 USA

All rights reserved. Copyright in this document is owned by Sun Microsystems, Inc.

Sun Microsystems, Inc. (SUN) hereby grants to you at no charge a nonexclusive, nontransferable, worldwide, limited license (without the right to sublicense) under SUN's intellectual property rights that are essential to practice the Java Card™ technology to use this document for internal evaluation purposes only. Other than this limited license, you acquire no right, title, or interest in or to the document and you shall have no right to use the document for productive or commercial use.

RESTRICTED RIGHTS LEGEND

Use, duplication, or disclosure by the U.S. Government is subject to restrictions of FAR 52.227-14(g)(2)(6/87) and FAR 52.227-19(6/87), or DFAR 252.227-7015(b)(6/95) and DFAR 227.7202-1(a).

SUN MAKES NO REPRESENTATIONS OR WARRANTIES ABOUT THE SUITABILITY OF THE SOFTWARE, EITHER EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT. SUN SHALL NOT BE LIABLE FOR ANY DAMAGES SUFFERED BY LICENSEE AS A RESULT OF USING, MODIFYING OR DISTRIBUTING THIS SOFTWARE OR ITS DERIVATIVES.

TRADEMARKS

Sun, Sun Microsystems, the Sun logo, Java, Java Card, SunDocs, and SunExpress are trademarks or registered trademarks of Sun Microsystems, Inc. in the United States and other countries. UNIX® is a registered trademark in the United States and other countries, exclusively licensed through X/Open Company, Ltd.

THIS PUBLICATION IS PROVIDED "AS IS" WITHOUT WARRANTY OF ANY KIND, EITHER EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, OR NON-INFRINGEMENT.

THIS PUBLICATION COULD INCLUDE TECHNICAL INACCURACIES OR TYPOGRAPHICAL ERRORS. CHANGES ARE PERIODICALLY ADDED TO THE INFORMATION HEREIN; THESE CHANGES WILL BE INCORPORATED IN NEW EDITIONS OF THE PUBLICATION. SUN MICROSYSTEMS, INC. MAY MAKE IMPROVEMENTS AND/OR CHANGES IN THE PRODUCT(S) AND/OR THE PROGRAM(S) DESCRIBED IN THIS PUBLICATION AT ANY TIME.

Contents

1. Introduction 1

2. Changes to Runtime Environment Specification 3

Summary of Changes 3

Detailed Changes 4

3.1 The Method install 4

6.1.2 Object Ownership 4

6.2.3 JCRE Privileges 5

7.6.3 Cleanup Responsibilities of the JCRE 5

7.7 Transient Objects and Global Arrays 6

10.1.4 Installer Behavior 6

11 API Constants 6

3. Changes to Application Programming Interface Specification 9

Summary of Changes 9

Detailed Changes 11

javacard.framework.Applet 11

javacard.framework.APDU 12

javacard.framework.OwnerPIN 13

javacard.framework.JCSystem 13

javacard.security.KeyPair 14

javacard.security.DSAKey 14

javacard.security.KeyBuilder 17

javacard.security.MessageDigest 17

javacard.security.Signature 18

javacardx.crypto.Cipher 21

4. Changes to Virtual Machine Specification 25

Summary of Changes 25

Detailed Changes 26

Section 2.2.1.1 Unsupported Features 26

Section 2.2.3.1 int 27

Section 2.2.4.5 Class Initialization 27

Section 3.1 Data Types and Values 28

Section 4.3.6 Token Assignment 29

Section 4.3.7.2 Classes and Interfaces 30

Section 4.3.7.3 Static Fields 30

Section 4.3.7.4 Static Methods and Constructors 31

Section 4.3.7.5 Instance Fields 31

Section 4.3.7.6 Virtual Methods 32

Section 5.3 Ownership 34

Section 5.4 Hierarchies Represented 34

Section 5.6.3 CONSTANT_Integer 35

Section 5.7 Classes and Interfaces 35

Section 5.8 Fields 36

Section 5.8 Fields 37

Section 5.9 Methods 38

Section 5.9 Methods 39

Section 6.2 Installation 39

Section 6.4 Directory Component 40

Section 6.7 Constant Pool Component 41

Section 6.9 Method Component 41

Section 6.9 Method Component 42

Section 6.9.1 exception_handler_info 43

Section 6.10 Static Field Component 47

Section 6.10 Static Field Component 48

Section 6.10 Static Field Component 49

Section 6.11 Reference Location Component 49

Section 6.13.1 class_descriptor_info 50

Section 6.13.2 field_descriptor_info 51

Section 6.13.3 method_descriptor_info 52

Chapter 7 Instruction Set, aastore 53

Chapter 7 Instruction Set, aastore, checkcast, interfaceof 53

Chapter 7 Instruction Set, getfield_<t>, getfield_<t>_this, getfield_<t>_w, putfield_<t>, putfield_<t>_this, putfield_<t>_w 55

Chapter 7 Instruction Set, getfield_<t>, getfield_<t>_this, getfield_<t>_w, putfield_<t>, putfield_<t>_this, putfield_<t>_w 56

Chapter 7 Instruction Set, invokespecial 56

Chapter 7 Instruction Set, invokestatic 58

Chapter 7 Instruction Set, invokevirtual 58

Chapter 7 Instruction Set, sshl, sshr 59

Chapter 7 Instruction Set, sushr 60

CHAPTER 1

Introduction

This document is the Release Notes for the Java Card™ 2.1.1 Specifications. The specifications include:

- *Java Card™ 2.1.1 Runtime Environment (JCRE) Specification*
- *Java Card™ 2.1.1 Application Programming Interface*
- *Java Card™ 2.1.1 Virtual Machine Specification*

These specifications describe the Java Card 2.1.1 platform standards of Java Card technology, providing the foundation for designing implementations of Java Card enabled devices and tools.

2

Changes to Runtime Environment Specification

The *Java Card™ 2.1.1 Runtime Environment (JCRE) Specification* describes the runtime environment required for interoperable Java Card applet execution.

Summary of Changes

Changes since the 2.1 release.

- Section 3.1 The Method `install` - clarifies that only one applet instance can be created on one invocation of the `install` method.
- Section 6.1.2 Object Ownership - Added detailed rules about group context behavior with respect to the ownership of objects and the `JCSystem.getPreviousContextAID` method
- Section 6.2.3 JCRE Privileges - Added clarification that the JCRE context can access fields and components of all objects including `CLEAR_ON_DESELECT` objects.
- Section 7.6.3 Cleanup Responsibilities of the JCRE - Added paragraph to allow card lockup on programmatic transaction abortion after object creation.
- Section 7.7 Transient Objects and Global Arrays - Renamed this section name to also include Global Arrays which do not participate in transactions.
- Section 10.1.4 Installer Behavior - Clarified JCRE cleanup requirements on installation failure.
- Section 11 API Constants - Added `ALG_RSA_NOPAD` constant in `javacardx.crypto.Cipher` class. Also introduced the constants associated with the new `javacardx.security.KeyPair` class.

Detailed Changes

The following shows all the changes since the 2.1 release.

3.1 The Method `install`

...

Typically, an applet creates various objects, initializes them with predefined values, sets some internal state variables, and calls either the `Applet.register()` method or the `Applet.register(byte[], short, byte)` method to specify the AID (applet IDentifier as defined in ISO 7816-5) to be used to select it. This installation is considered successful when the call to the `Applet.register` method completes without an exception. The installation is deemed unsuccessful if the `install` method does not call the `Applet.register` method, or if an exception is thrown from within the `install` method prior to the `Applet.register` method being called, or if the `Applet.register` method throws an exception. If the installation is unsuccessful, the JCRE shall perform all cleanup when it regains control. That is, all persistent objects shall be returned to the state they had prior to calling the `install` method. If the installation is successful, the JCRE can mark the applet as available for selection.

Only one applet instance can be successfully registered each time the JCRE calls the `Applet.install` method.

6.1.2 Object Ownership

When a new object is created, it is associated with the currently active context. But the object is *owned* by the applet instance within the currently active context when the object is instantiated. An object is owned by an applet instance, or by the JCRE.

The combined rules of group context and object ownership within the firewall are:

1. Every object is owned by an applet instance (or the JCRE). An applet instance is identified by its AID. When executing in an instance method of an object (or a static class method called from within), the object owner's context is the currently active context.

2. Every applet instance belongs to a group context. All applet instances from the same package belong to the same context.

Assume that applet A1 and A2 are in the same package, and applet B0 is in another package. A1 and A2 therefore belong to the same group context A, whereas B0 belongs to When the current context is that of applet A1, and a method m1 in another object owned by applet A1 is invoked, no group context switch occurs. When the current context is that of applet A1, and a method m2 in an object owned by applet A2 is invoked, again no context switch occurs (in spite of the object “owner” change), and no firewall restrictions apply. However, if the method m2 now calls a method m0 in an object owned by applet B0, firewall restrictions apply and a context switch shall occur. Upon return to method m2 from the method m0, the context of applet A2 is restored.

In the flow sequence mentioned in the last paragraph, when the method in object owned by applet A1 calls the method m2 in the object owned by applet A2, the context does not change but the owner of the object does change. Furthermore, applet A2 is the owner of the object when the context switches from A to B. Therefore, if the `JCSystem.getPreviousContextAID` method is called from method m0 within context B, the AID of applet A2 shall be returned.

Also note that if the `JCSystem.getPreviousContextAID` method is now called from method m2 after the return from method m0 in context B, the AID of the applet which called into context A (or null if JCRE) is returned and not the AID of applet B0.

6.2.3 JCRE Privileges

...

...During such an invocation, a context switch occurs from the JCRE context to the context of the applet that owns X.

Again, because it is the “system” context, the JCRE context can access fields and components of any object on the card including `CLEAR_ON_DESELECT` transient objects owned by the currently selected applet.

7.6.3 Cleanup Responsibilities of the JCRE

Object instances created during the transaction that is being aborted can be deleted only if references to these deleted objects can no longer be used to access these objects. The JCRE shall ensure that a reference to an object created during the aborted transaction is equivalent to a `null` reference.

Alternatively, programmatic abortion after creating objects within the transaction can be deemed to be a programming error. When this occurs, the JCRE may, to ensure the security of the card and to avoid heap space loss, lock up the card session to force tear/reset processing.

7.7 Transient Objects and Global Arrays

Only updates to persistent objects participate in the transaction. Updates to transient objects and global arrays are never undone, regardless of whether or not they were "inside a transaction."

10.1.4 Installer Behavior

...

The JCRE shall guarantee that an applet will *not* be deemed successfully installed if:

- ...
- a class in the applet package is found to contain more package visible virtual methods or instance fields than the limitations enumerated in section 2.2.4.2 of the *Java Card™ 2.1.1 Virtual Machine Specification*.
- a reset or power fail occurs while executing the applet's `install` method and before successful return from the `Applet.register` method (see Section 3.1).
- the applet's `install` method throws an exception before successful return from the `Applet.register` method (see Section 3.1).

When applet installation is unsuccessful, the JCRE shall guarantee that objects created during the execution of the `install` method, or by the JCRE on it's behalf - initialized static arrays - can never be accessed by any applet on the card.

11 API Constants

...

Class `javacard.security.KeyPair`

```
public static final byte ALG_RSA = 1;
public static final byte ALG_RSA_CRT = 2;
public static final byte ALG_DSA = 3;
```

```
...
```

Class `javacardx.crypto.Cipher`

```
...
public static final byte ALG_RSA_ISO9796 = 11;
public static final byte ALG_RSA_NOPAD = 12;
public static final byte MODE_DECRYPT = 1;
public static final byte MODE_ENCRYPT = 2;
```

3

Changes to Application Programming Interface Specification

The *Java Card™ 2.1.1 Application Programming Interface* defines a set of classes upon which Java Card Applets may be constructed.

Summary of Changes

Changes since since the 2.1 release.

- `javacard.framework.Applet.install` method clarifies that only one instance can be successfully created at one time.
- `javacard.framework.Applet.register` methods clarify behavior of method when called from a previously registered applet.
- `javacard.framework.Applet.register` methods clarify behavior of method when called from outside JCRE initiated `install()`.
- `javacard.framework.Applet.register` method now uses “Java Card name” instead of the term “Applet subclass AID bytes” alongwith a description.

- `javacard.framework.APDU.sendBytesLong` method now throws a `SecurityException` when the input array is not accessible to caller.
- `javacard.framework.OwnerPIN` constructor throws a `PINException` if the `tryLimit` parameter is `<=0`.
- `javacard.framework.JCSystem.abortTransaction` method warns of a possible lock up of the card session if objects were created.

- `javacard.security.KeyPair` class has been added.

- `javacard.security.DSAKey` class now correctly associates P,Q and G nomenclature with prime, subprime and base parameters respectively.
- `javacard.security.KeyBuilder.buildKey` method clarifies that default `KeyEncryption` support is allowed. It also clarifies that the returned keys are uninitialized.

- `javacard.security.MessageDigest` class clarifies the behavior on tear and reset events.
- `javacard.security.MessageDigest.update` method clarifies the warning about use of non-block aligned input data.

- `javacard.security.Signature` class clarifies the behavior on tear and reset events.
- `javacard.security.Signature class clarifies the algorithms required in the ALG_RSA_SHA_PKCS1 and ALG_RSA_MD5_PKCS1 selectors.`
- `javacard.security.Signature.update` method clarifies the warning about use of non-block aligned input data.
- `javacard.security.Signature.verify/sign` methods clarify the state of the Signature object upon completion.

- `javacardx.crypto.Cipher` class clarifies the behavior on tear and reset events.
- `javacardx.crypto.Cipher class clarifies the algorithm required in the ALG_RSA_PKCS1 selector.`
- `javacardx.crypto.Cipher class adds the new algorithm selector ALG_RSA_NOPAD.`
- `javacardx.crypto.Cipher.update` method clarifies the warning about use of non-block aligned input data.
- `javacardx.crypto.Cipher.doFinal` method clarifies the state of the `Cipher` object upon completion.
- `javacardx.crypto.Cipher.update/doFinal` methods warns about incorrect results when input/output buffer overlaps such that the output may overwrite input before it is processed.
- `javacardx.crypto.Cipher.update/doFinal` methods warns about incorrect results when non-block aligned input is used.
- `javacardx.crypto.Cipher.doFinal` method may throw an exception if bad padding data is detected.
- Fixed numerous typographical errors. Added more specific references to JCRE spec

Detailed Changes

The following shows all the changes since the 2.1 release.

javacard.framework.Applet

```
public static void install(byte bArray[],
                           short bOffset,
                           byte bLength)
       throws javacard.framework.ISOException
```

To create an instance of the Applet subclass, the JCRE will call this static method first.

The applet should perform any necessary initializations and must call one of the `register()` methods. Only one Applet instance can be successfully registered from within this install. The installation is considered successful when the call to `register()` completes without an exception. The installation is deemed unsuccessful if the install method does not call a `register()` method, or if an exception is thrown from within the install method prior to the call to a `register()` method, or if every call to the `register()` method results in an exception. If the installation is unsuccessful, the JCRE must perform all the necessary clean up when it receives control. Successful installation makes the applet instance capable of being selected via a SELECT APDU command.

...

```
protected final void register()
       throws javacard.framework.SystemException
```

This method is used by the applet to register `this` applet instance with the JCRE and to assign the Java Card name of the applet as its instance AID bytes. One of the `register()` methods must be called from within `install()` to be registered with the JCRE. See *Java Card 2.1 Runtime Environment (JCRE) Specification*, section 3.1 for details.

Note:

- *The phrase "Java card name of the applet" is a reference to the* `AID[AID_length]` *item in the* `applets[]` *item of the* `applet_component`*, as documented in Section 6.5 Applet Component in the Java Card Virtual Machine 2.1 Specification.*

Throws:

SystemException – with the following reason codes:

- `SystemException.ILLEGAL_AID` if the `Applet` subclass AID bytes are in use or if the applet instance has previously successfully registered with the JCRE via one of the `register()` methods or if a JCRE initiated `install()` method execution is not in progress.

```
protected final void register(byte bArray[],
                              short bOffset,
                              byte bLength)
      throws javacard.framework.SystemException
```

This method is used by the applet to register this applet instance with the JCRE and assign the specified AID bytes as its instance AID bytes. One of the `register()` methods must be called from within `install()` to be registered with the JCRE. See *Java Card Runtime Environment (JCRE) 2.1 Specification* for details.

Parameters:

...

Throws:

`SystemException` – with the following reason code:

- `SystemException.ILLEGAL_VALUE` if the `bLength` parameter is less than 5 or greater than 16.
- `SystemException.ILLEGAL_AID` if the specified instance AID bytes are in use or if the RID portion of the AID bytes in the `bArray` parameter does not match the RID portion of the Java Card name of the applet or if the applet instance has previously successfully registered with the JCRE via one of the `register()` methods or if a JCRE initiated `install()` method execution is not in progress.

Note:

- *The phrase "Java card name of the applet" is a reference to the* `AID[AID_length]` *item in the* `applets[]` *item of the* `applet_component`*, as documented in Section 6.5 Applet Component in the Java Card Virtual Machine 2.1 Specification.*

javacard.framework.APDU

```
public void sendBytesLong(byte outData[],
                          short bOff,
                          short len) throws
      javacard.framework.APDUException
```

...

Throws:

SecurityException – if the outData byte array is not accessible in the caller's context.

APDUException

...

javacard.framework.OwnerPIN

```
public OwnerPIN(byte tryLimit,
                byte maxPINSize)
         throws javacard.framework.PINException
```

Constructor. Allocates a new `PIN` instance with validated flag set to `false`.

Parameters:

`tryLimit` – the maximum number of times an incorrect PIN can be presented. `tryLimit` must be >=1.

maxPINSize - the maximum allowed PIN size. maxPINSize must be >=1.

Throws:

PINException – with the following reason codes:

- `PINException.ILLEGAL_VALUE` if `tryLimit` parameter is less than 1.
- `PINException.ILLEGAL_VALUE` if `maxPINSize` parameter is less than 1.

javacard.framework.JCSystem

```
public static native void abortTransaction()
                    throws javacard.framework.TransactionException
```

Aborts the atomic transaction. The contents of the commit buffer is discarded.

Notes:

- *Do not call this method from within a transaction which creates new objects because the JCRE may not recover the heap space used by the new object instances.*
- *Do not call this method from within a transaction which creates new objects because the JCRE may, to ensure the security of the card and to avoid heap space loss, lock up the card session to force tear/reset processing.*
- *The JCRE ensures that any variable of reference type which references an object instantiated from within this aborted transaction is equivalent to a null reference.*

...

javacard.security.KeyPair

This is a new key pair encapsulation class that includes a key generation facility. This class was previously called "`AsymmetricKeyPair`" in the Alpha release. The key generation method "`generateKeyData()`" has also been renamed to "`genKeyPair()`".

javacard.security.DSAKey

```
public abstract void setP(byte buffer[],
                          short offset,
                          short length)
                   throws javacard.security.CryptoException
```

Sets the prime parameter value of the key. The plaintext data format is big-endian and right-aligned (the least significant bit is the least significant bit of last byte). Input prime parameter data is copied into the internal representation.

Parameters:

buffer – the input buffer

offset – the offset into the input buffer at which the prime parameter value begins

length – the length of the prime parameter value

...

Note:

- *If the key object implements the* `javacardx.crypto.KeyEncryption` *interface and the* `Cipher` *object specified via* `setKeyCipher()` *is not* `null`*, the prime parameter value is decrypted using the* `Cipher` *object.*

```
public abstract void setQ(byte buffer[],
                          short offset,
                          short length) throws
                 javacard.security.CryptoException
```

Sets the subprime parameter value of the key. The plaintext data format is big-endian and right-aligned (the least significant bit is the least significant bit of last byte). Input subprime parameter data is copied into the internal representation.

Parameters:

buffer – the input buffer

offset – the offset into the input buffer at which the subprime parameter value begins

length – the length of the subprime parameter value

...

Note:

- *If the key object implements the* `javacardx.crypto.KeyEncryption` *interface and the* `Cipher` *object specified via* `setKeyCipher()` *is not* `null`*, the subprime parameter value is decrypted using the* `Cipher` *object.*

```
public abstract void setG(byte buffer[],
                          short offset,
                          short length)
                   throws javacard.security.CryptoException
```

Sets the base parameter value of the key. The plaintext data format is big-endian and right-aligned (the least significant bit is the least significant bit of last byte). Input base parameter data is copied into the internal representation.

Parameters:

buffer – the input buffer

offset – the offset into the input buffer at which the base parameter value begins

length – the length of the base parameter value

...

Note:

- *If the key object implements the* `javacardx.crypto.KeyEncryption` *interface and the* `Cipher` *object specified via* `setKeyCipher()` *is not* `null`*, the base parameter value is decrypted using the* `Cipher` *object.*

```
public abstract short getP(byte buffer[],
                           short offset)
```

Returns the prime parameter value of the key in plain text. The data format is big-endian and right-aligned (the least significant bit is the least significant bit of last byte).

Parameters:

`buffer` – the output buffer

`offset` – the offset into the output buffer at which the prime parameter value begins

Returns:

the byte length of the prime parameter value returned

```
public abstract short getQ(byte buffer[],
                           short offset)
```

Returns the subprime parameter value of the key in plain text. The data format is big-endian and right-aligned (the least significant bit is the least significant bit of last byte).

Parameters:

`buffer` – the output buffer

`offset` – the offset into the output buffer at which the subprime parameter value begins

Returns:

the byte length of the subprime parameter value returned

```
public abstract short getG(byte buffer[],
                           short offset)
```

Returns the base parameter value of the key in plain text. The data format is big-endian and right-aligned (the least significant bit is the least significant bit of last byte).

Parameters:

`buffer` – the output buffer

`offset` – the offset into the output buffer at which the base parameter value begins

Returns:

the byte length of the base parameter value returned

javacard.security.KeyBuilder

```
public static key buildKey(byte keytype,
                   short keyLength,
                   boolean keyEncryption)
             throws javacard.security.CryptoException
```

Creates uninitialized cryptographic keys for signature and cipher algorithms. Instances created by this method may be the only key objects used to initialize instances of `Signature` and `Cipher`. Note that the object returned must be cast to their appropriate key type interface.

Parameters:

`keyType` – the type of key to be generated. Valid codes listed in TYPE.. constants.

`keyLength` – the key size in bits. The valid key bit lengths are key type dependent. See above.

`keyEncryption` – if `true` this boolean requests a key implementation which implements the `javacardx.crypto.KeyEncryption` interface. The key implementation returned may implement the `javacardx.crypto.KeyEncryption` interface even when this parameter is false.

...

javacard.security.MessageDigest

```
public abstract class MessageDigest
extends java.lang.Object
```

The `MessageDigest` class is the base class for hashing algorithms. Implementations of MessageDigest algorithms must extend this class and implement all the abstract methods.

A tear or card reset event resets a `MessageDigest` object to the initial state (state upon construction).

...

```
 public abstract void update(byte inBuff[],
                             short inOffset,
                             short inLength)
```

Accumulates a hash of the input data. This method requires temporary storage of intermediate results. In addition, if the input data length is not block aligned (multiple of block size) then additional internal storage may be allocated at this time to store a partial input data block. This may result in additional resource

consumption and/or slow performance. This method should only be used if all the input data required for the hash is not available in one byte array. The doFinal() method is recommended whenever possible.

...

javacard.security.Signature

```
public abstract class Signature
extends java.lang.Object
```

The `Signature` class is the base class for Signature algorithms. Implementations of Signature algorithms must extend this class and implement all the abstract methods.

The term "pad" is used in the public key signature algorithms below to refer to all the operations specified in the referenced scheme to transform the message digest into the encryption block size.

A tear or card reset event resets an initialized `Signature` object to the state it was in when previously initialized via a call to `init()`.

Note:

- *On a tear or card reset event, the DES and triple DES algorithms in outer CBC mode reset the initial vector(IV) to 0. The initial vector(IV) can be re-initialized using the* `init(Key, byte, byte[], short, short)` *method.*

...

`public static final byte` **`ALG_RSA_SHA_PKCS1`**

Signature algorithm `ALG_RSA_SHA_PKCS1` encrypts the 20 byte SHA digest using RSA. The digest is padded according to the PKCS#1 (v1.5) scheme.

Note:

- *The encryption block(EB) during signing is built as follows:*

  EB = 00 || 01 || PS || 00 || T

  :: where T is the DER encoding of :

  digestInfo ::= SEQUENCE {

  digestAlgorithm AlgorithmIdentifier of SHA-1,

  digest OCTET STRING

  }

  :: PS is an octet string of length k-3-||T|| with value FF. The length of PS must be at least 8 octets.

:: k is the RSA modulus size.

`public static final byte` **`ALG_RSA_MD5_PKCS1`**

Signature algorithm `ALG_RSA_MD5_PKCS1` encrypts the 16 byte MD5 digest using RSA. The digest is padded according to the PKCS#1 (v1.5) scheme.

Note:

The encryption block(EB) during signing is built as follows:

< EB = 00 || 01 || PS || 00 || T

:: where T is the DER encoding of :

digestInfo ::= SEQUENCE {

digestAlgorithm AlgorithmIdentifier of MD5,

digest OCTET STRING

}

:: PS is an octet string of length k-3-||T|| with value FF. The length of PS must be at least 8 octets.

:: k is the RSA modulus size.

`public static final byte` **`ALG_RSA_RIPEMD160_PKCS1`**

Signature algorithm `ALG_RSA_RIPEMD160_PKCS1` encrypts the 16 byte RIPEMD160 digest using RSA. The digest is padded according to the PKCS#1 (v1.5) scheme.

Note:

The encryption block(EB) during signing is built as follows:

< EB = 00 || 01 || PS || 00 || T

:: where T is the DER encoding of :

digestInfo ::= SEQUENCE {

digestAlgorithm AlgorithmIdentifier of RIPEMD160,

digest OCTET STRING

}

:: PS is an octet string of length k-3-| |T| | with value FF. The length of PS must be at least 8 octets.

:: k is the RSA modulus size.

```
public abstract void update(byte inBuff[],
                            short inOffset,
                            short inLength)
                     throws javacard.security.CryptoException
```

Accumulates a signature of the input data. This method requires temporary storage of intermediate results. In addition, if the input data length is not block aligned (multiple of block size) then additional internal storage may be allocated at this time to store a partial input data block. This may result in additional resource consumption and/or slow performance. This method should only be used if all the input data required for signing/verifying is not available in one byte array. The `sign()` or `verify()` method is recommended whenever possible.

...

```
public abstract short sign(byte inBuff[],
                           short inOffset,
                           short inLength,
                           byte sigBuff[],
                           short sigOffset)
                     throws javacard.security.CryptoException
```

Generates the signature of all/last input data. A call to this method also resets this `Signature` object to the state it was in when previously initialized via a call to `init()`. That is, the object is reset and available to sign another message.

Note:

- *DES and triple DES algorithms in outer CBC mode reset the initial vector(IV) to 0. The initial vector(IV) can be re-initialized using the* `init(Key, byte, byte[], short, short)` *method.*

...

```
public abstract boolean verify(byte inBuff[],
                               short inOffset,
                               short inLength,
                               byte sigBuff[],
                               short sigOffset,
                               short sigLength)
                     throws javacard.security.CryptoException
```

Verifies the signature of all/last input data against the passed in signature. A call to this method also resets this `Signature` object to the state it was in when previously initialized via a call to `init()`. That is, the object is reset and available to verify another message.

Note:

- *DES and triple DES algorithms in outer CBC mode reset the initial vector(IV) to 0. The initial vector(IV) can be re-initialized using the* `init(Key, byte, byte[], short, short)` *method.*

...

javacardx.crypto.Cipher

```
public abstract class Cipher
extends java.lang.Object
```

The `Cipher` class is the abstract base class for Cipher algorithms. Implementations of Cipher algorithms must extend this class and implement all the abstract methods.

The term "pad" is used in the public key cipher algorithms below to refer to all the operations specified in the referenced scheme to transform the message block into the cipher block size.

The aysmmetric key algorithms encrypt using either a public key (to cipher) or a private key (to sign). In addition they decrypt using the either a private key (to decipher) or a public key (to verify).

A tear or card reset event resets an initialized `Cipher` object to the state it was in when previously initialized via a call to `init()`.

Note:

- *On a tear or card reset event, the DES and triple DES algorithms in outer CBC mode reset the initial vector(IV) to 0. The initial vector(IV) can be re-initialized using the* `init(Key, byte, byte[], short, short)` *method.*

...

```
public static final byte ALG_RSA_PKCS1
```

Cipher algorithm `ALG_RSA_PKCS1` provides a cipher using RSA. Input data is padded according to the PKCS#1 (v1.5) scheme.

Note:

- *This algorithm is only suitable for messages of limited length. The total number of input bytes processed may not be more than k-11, where k is the RSA key's modulus size in bytes.*

- *The encryption block(EB) during encryption with a Public key is built as follows:*

  EB = 00 || 02 || PS || 00 || M

  :: M (input bytes) is the plaintext message

  :: PS is an octet string of length k-3-||M|| of pseudo random nonzero octets. The length of PS must be at least 8 octets.

  :: k is the RSA modulus size.
- *The encryption block(EB) during encryption with a Private key (used to compute signatures when the message digest is computed off-card) is built as follows:*

  EB = 00 || 01 || PS || 00 || D

  :: D (input bytes) is the DER encoding of the hash computed elsewhere with an algorithm ID prepended if appropriate

  :: PS is an octet string of length k-3-||D|| with value FF. The length of PS must be at least 8 octets.

  :: k is the RSA modulus size.

...

`public static final byte` **`ALG_DES_ECB_NOPAD`**

Cipher algorithm `ALG_DES_ECB_NOPAD` provides a cipher using DES in ECB mode. This algorithm does not pad input data. If the input data is not (8 byte) block aligned it throws `CryptoException` with the reason code `ILLEGAL_USE`.

```
public abstract short doFinal(byte inBuff[],
                              short inOffset,
                              short inLength,
                              byte outBuff[],
                              short outOffset)
                       throws javacard.security.CryptoException
```

Generates encrypted/decrypted output from all/last input data. A call to this method also resets this Cipher object to the state it was in when previously initialized via a call to `init()`. That is, the object is reset and available to encrypt or decrypt (depending on the operation mode that was specified in the call to `init()`) more data.

Note:

- *When using block-aligned data (multiple of block size), if the input buffer, inBuff and the output buffer, outBuff are the same array, then the output data area must not partially overlap the input data area such that the input data is modified before it is used; if* `inBuff==outBuff` *and* `inOffset < outOffset < inOffset+inLength`*, incorrect output may result.*
- *When non-block aligned data is presented as input data, no amount of input and output buffer data overlap is allowed; if* `inBuff==outBuff` *and* `outOffset < inOffset+inLength`*, incorrect output may result.*
- *DES and triple DES algorithms in outer CBC mode reset the initial vector(IV) to 0. The initial vector(IV) can be re-initialized using the* `init(Key, byte, byte[], short, short)` *method.*
- *...*

...

Throws:

CryptoException with the following reason codes:

- `CryptoException.UNINITIALIZED_KEY` if key not initialized.
- `CryptoException.INVALID_INIT` if this Cipher object is not initialized.
- `CryptoException.ILLEGAL_USE` if this Cipher algorithm does not pad the message and the message is not block aligned or if the input message length is not supported or if the decrypted data is not bounded by appropriate padding bytes.

...

```
public abstract short update(byte inBuff[],
                             short inOffset,
                             short inLength,
                             byte outBuff[],
                             short outOffset) throws
javacard.security.CryptoException
```

Generates encrypted / decrypted output from input data. This method requires temporary storage of intermediate results. In addition, if the input data length is not block aligned (multiple of block size) then additional internal storage may be allocated at this time to store a partial input data block. This may result in additional resource consumption and / or slow performance. This method should only be used if all the input data required for the cipher is not available in one byte array. The `doFinal()` method is recommended whenever possible.

Notes:

- *When using block-aligned data (multiple of block size), if the input buffer, inBuff and the output buffer, outBuff are the same array, then the output data area must not partially overlap the input data area such that the input data is modified before it is used; if* `inBuff==outBuff` *and* `inOffset < outOffset < inOffset+inLength`*, incorrect output may result.*

- *When non-block aligned data is presented as input data, no amount of input and output buffer data overlap is allowed; if* `inBuff==outBuff` *and* `outOffset < inOffset+inLength`*, incorrect output may result.*
- *...*

4

Changes to Virtual Machine Specification

The *Java Card™ 2.1.1 Virtual Machine Specification* defines the required behavior of a virtual machine for a Java Card enabled device.

Summary of Changes

The following is an enumeration of major changes since the 2.1 release.

- Section 2.2.1.1 Unsupported Features, Access Control in Java Packages - added two constraints that are consistent with the specification. They do not represent a change in functionality.
- Section 2.2.4.5 Class Initialization - added specification of bytecodes supported in class initialization methods.
- Sections 3.1 Data Types and Values, 5.6.3 CONSTANT_Integer, and 6.10 Static Field Component - added specification of boolean values.
- Section 5.4 Hierarchies Represented - added section to clarify export file format design.
- Section 6.4 Directory Component - removed constraint for AID values for custom components.
- Chapter 7 Instruction Set, get/putfield, invokespecial, and invokevirtual instructions - corrected qualification of access to protected fields and methods. Please note that these corrections differ from the parallel statements in both the *Java Virtual Machine Specification* and the *Java Virtual Machine Specification, second edition*. Instead they reflect what will be the content of the *Java Virtual Machine Specification, third edition*, and the behavior of current Java virtual machine implementations.

- Chapter 7 Instruction Set, invokestatic - removed qualification of access to protected static methods.
- Added many clarifications and descriptions based on comments and questions from licensees. These do not change the definition of the specification.

Detailed Changes

The following shows all the changes since the 2.1 release.

Section 2.2.1.1 Unsupported Features

Corrected - Added access control constraints

Access Control in Java Packages

The Java Card language subset supports the package access control defined in the Java language. However, the cases that are not supported are as follows.

- If a class implements a method with `package` access visibility, a subclass cannot override the method and change the access visibility of the method to `protected` or `public`.
- An interface that is defined with `package` access visibility cannot be extended by an interface with `public` access visibility.
- A `public` or `protected` method in a `public` class cannot contain a formal parameter of type reference to a `package-visible` class.
- A `package-visible` class that is extended by a `public` class cannot define any `public` or `protected` methods.

Original -

Access Control in Java Packages

The Java Card language subset supports the package access control defined in the Java language. However, there are two cases that are not supported.

- If a class implements a method with `package` access visibility, a subclass cannot override the method and change the access visibility of the method to `protected` or `public`.
- An interface that is defined with `package` access visibility cannot be extended by an interface with `public` access visibility.

Section 2.2.3.1 int

Corrected - Added description of equal results constraint

The `int` keyword and 32-bit integer data types need not be supported in a Java Card implementation. A Java Card virtual machine that does not support the `int` data type will reject programs which use the `int` data type or 32-bit intermediate values.

The result of an arithmetic expression produced by a Java Card virtual machine must be equal to the result produce by a Java virtual machine, regardless of the input values. A Java Card virtual machine that does not support the `int` data type must reject expressions that could produce a different result.

Original -

The `int` keyword and 32-bit integer data types need not be supported in a Java Card implementation. A Java Card virtual machine that does not support the `int` data type will reject programs which use the `int` data type or 32-bit intermediate values.

Section 2.2.4.5 Class Initialization

Corrected - Added specific bytecodes supported in class initialization methods

There is limited support for initialization of static field values in `<clinit>` methods. Static fields of applets may only be initialized to primitive compile-time constant values, or arrays of primitive compile-time constants. Static fields of user libraries may only be initialized to primitive compile-time constant values. Primitive constant data types include `boolean`, `byte`, `short`, and `int`.

Given Java source files that adhere to these language-level constraints on static field initialization, it is expected that reasonable Java compilers will produce only the following bytecodes:

load a value on the stack: `iconst_[0-5]`, `[b|s]ipush, ldc, aconst_null`

create an array: `newarray([byte|short|boolean|int] )`

duplicate items on the stack: `dup`

store values in arrays or static fields: `[b|s]astore, putstatic`

return from method: `return`

Original -

There is limited support for initialization of static field values in `<clinit>` methods. Static fields of applets may only be initialized to primitive compile-time constant values, or arrays of primitive compile-time constants. Static fields of user libraries may only be initialized to primitive compile-time constant values. Primitive constant data types include `boolean`, `byte`, `short`, and `int`.

Section 3.1 Data Types and Values

Corrected - Added specification of boolean values

The Java Card virtual machine supports the same two kinds of data types as the Java Virtual Machine: *primitive types* and *reference types*. Likewise, the same two kinds of values are used: *primitive values* and *reference values*.

The primitive data types supported by the Java Card virtual machine are the *numeric types* and the `returnAddress` type. The numeric types consist only of the *integral types*:

- `byte`, whose values are 8-bit signed two's complement integers
- `short`, whose values are 16-bit signed two's complement integers

Some Java Card virtual machine implementations may also support an additional integral type:

- `int`, whose values are 32-bit signed two's complement integers

Support for the `boolean` type is identical to that in the Java Virtual Machine. The value 1 is used to represent *true* and the value of 0 is used to represent *false*.

Support for `reference` types is identical to that in the Java Virtual Machine.

Original -

The Java Card virtual machine supports the same two kinds of data types as the Java Virtual Machine: *primitive types* and *reference types*. Likewise, the same two kinds of values are used: *primitive values* and *reference values*.

The primitive data types supported by the Java Card virtual machine are the *numeric types* and the `returnAddress` type. The numeric types consist only of the *integral types*:

- `byte`, whose values are 8-bit signed two's complement integers
- `short`, whose values are 16-bit signed two's complement integers

Some Java Card virtual machine implementations may also support an additional integral type:

- `int`, whose values are 32-bit signed two's complement integers

Support for `reference` types is identical to that in the Java Virtual Machine.

Section 4.3.6 Token Assignment

Corrected - Added concept of API owner instead of developer

Tokens for an API are assigned by the API's owner and published in the package `export` file(s) for that API. Since the name-to-token mappings are published, an API owner may choose any order for tokens (subject to the constraints listed below).

Original -

Tokens for an API are assigned by the API's developer and published in the package `export` file(s) for that API. Since the name-to-token mappings are published, an API developer may choose any order for tokens (subject to the constraints listed below).

Section 4.3.7.2 Classes and Interfaces

Corrected - Reorganized description for clarity

All externally visible classes in a package are assigned public *class tokens*. Class token values must be in the range from 0 to 255, inclusive. The tokens for all the public classes in a package are numbered consecutively starting at zero. The ordering of class tokens is not specified.

Package-visible classes are not assigned tokens.

Original -

All externally visible classes in a package are assigned public *class tokens*. Package-visible classes are not assigned tokens. Class token values must be in the range from 0 to 255, inclusive. The tokens for all the public classes in a package are numbered consecutively starting at zero. The ordering of class tokens is not specified.

Section 4.3.7.3 Static Fields

Corrected - Reorganized description for clarity

All externally visible static fields in a package are assigned public *static field tokens*. The tokens for all other externally visible static fields in a class are numbered consecutively starting at zero. Static fields token values must be in the range from 0 to 255, inclusive. The ordering of static field tokens is not specified.

Package-visible and private static fields are not assigned tokens. In addition, no tokens are assigned for final static fields that are initialized to primitive, compile-time constants, as these fields are never represented as fields in CAP files.

Original -

All externally visible static fields in a package are assigned public *static field tokens*. Package-visible and private static fields are not assigned tokens. No tokens are assigned for final static fields that are initialized to primitive, compile-time constants, as these fields are never linked on a device. The tokens for all other externally visible static fields in a class are numbered consecutively starting at zero. Static fields token values must be in the range from 0 to 255, inclusive. The ordering of static field tokens is not specified.

Section 4.3.7.4 Static Methods and Constructors

Corrected - Reorganized description for clarity, added mention of constructors

All externally visible static methods and constructors in a package are assigned public *static method tokens*. Constructors are included in this category because they are statically bound. Static method token values must be in the range from 0 to 255, inclusive. The tokens for all the externally visible static methods and constructors in a class are numbered consecutively starting at zero. The ordering of static method tokens is not specified.

Package-visible and private static methods as well as package-visible and private constructors are not assigned tokens.

Original -

All externally visible static methods in a package are assigned public *static method tokens*, including statically bound instance methods. Static method token values must be in the range from 0 to 255, inclusive. Package-visible and private static methods are not assigned tokens. The tokens for all the externally visible static methods in a class are numbered consecutively starting at zero. The ordering of static method tokens is not specified.

Section 4.3.7.5 Instance Fields

Corrected - Reorganized description for clarity

All instance fields defined in a package are assigned either public or private *instance field tokens*. The scope of a set of instance field tokens is limited to the class that declares the instance fields, not including the fields declared by superclasses of that class.

Instance field token values must be in the range from 0 to 255, inclusive. Public and private tokens for instance fields are assigned from the same namespace. The tokens for all the instance fields in a class are numbered consecutively starting at zero, except that the token after an `int` field is skipped and the token for the following field is numbered two greater than the token of the `int` field.

Within a class, tokens for externally visible fields must be numbered less than the tokens for package and private fields. For public tokens, the tokens for reference type fields must be numbered greater than the tokens for primitive type fields. For private tokens, the tokens for reference type fields must be numbered less than the tokens for primitive type fields. Beyond that, the ordering of instance field tokens in a class is not specified.

Original -

All instance fields defined in a package are assigned either public or private *instance field tokens*. Instance field token values must be in the range from 0 to 255, inclusive. Public and private tokens for instance fields are assigned from the same namespace. The tokens for all the instance fields in a class are numbered consecutively starting at zero, except that the token after an `int` field is skipped and the token for the following field is numbered two greater than the token of the `int` field. Tokens for externally visible fields must be numbered less than the tokens for package and private fields. For public tokens, the tokens for reference type fields must be numbered greater than the tokens for primitive type fields. For private tokens, the tokens for reference type fields must be numbered less than the tokens for primitive type fields. Beyond that the ordering of instance field tokens in a class is not specified.

Section 4.3.7.6 Virtual Methods

Corrected - Reorganized description for clarity

Virtual methods are instance methods that are resolved dynamically. The set includes all public, protected and package-visible instance methods. Private instance methods and all constructors are not virtual methods, but instead are resolved statically during compilation.

All virtual methods defined in a package are assigned either public or private *virtual method tokens*. Virtual method token values must be in the range from 0 to 127, inclusive. Public and private tokens for virtual methods are assigned from different namespaces. The high bit of the byte containing a virtual method token is set to one if the token is a private token.

Public tokens for the externally visible (public or protected) introduced virtual methods in a class are numbered consecutively starting at one greater than the highest numbered public virtual method token of the class's superclass. If a method overrides a method implemented in the class's superclass, that method is assigned

the same token number as the method in the superclass. The high bit of the byte containing a public virtual method token is always set to zero, to indicate it is a public token. The ordering of public virtual method tokens in a class is not specified.

Private virtual method tokens are assigned to package-visible virtual methods. They are assigned differently from public virtual method tokens. If a class and its superclass are defined in the same package, the tokens for the package-visible introduced virtual methods in that class are numbered consecutively starting at one greater than the highest numbered private virtual method token of the class's superclass. If the class and its superclass are defined in different packages, the tokens for the package-visible introduced virtual methods in that class are numbered consecutively starting at zero. If a method overrides a method implemented in the class's superclass, that method uses the same token number as the method in the superclass. The definition of the Java programming language specifies that overriding a package-visible virtual method is only possible if both the class and its superclass are defined in the same package. The high bit of the byte containing a virtual method token is always set to one, to indicate it is a private token. The ordering of private virtual method tokens in a class is not specified.

Original -

All virtual methods defined in a package are assigned either public or private *virtual method tokens*. Virtual method token values must be in the range from 0 to 127, inclusive. Public and private tokens for virtual methods are assigned from different namespaces. The high bit of the byte containing a virtual method token is set to one if the token is a private token.

Public tokens for the externally visible introduced virtual methods in a class are numbered consecutively starting at one greater than the highest numbered public virtual method token of the class's superclass. If a method overrides a method implemented in the class's superclass, that method uses the same token number as the method in the superclass. The high bit of the byte containing a public virtual method token is always set to zero, to indicate it is a public token. The ordering of public virtual method tokens in a class is not specified.

Private virtual method tokens are assigned differently from public virtual method tokens. If a class and its superclass are defined in the same package, the tokens for the package-visible introduced virtual methods in that class are numbered consecutively starting at one greater than the highest numbered private virtual method token of the class's superclass. If the class and its superclass are defined in different packages, the tokens for the package-visible introduced virtual methods in that class are numbered consecutively starting at zero. If a method overrides a method implemented in the class's superclass, that method uses the same token number as the method in the superclass. The definition of the Java programming language specifies that overriding a package-visible virtual method is only possible if both the class and its superclass are defined in the same package. The high bit of

the byte containing a virtual method token is always set to one, to indicate it is a private token. The ordering of private virtual method tokens in a class is not specified.

Section 5.3 Ownership

Corrected - Added section to clarify export file ownership

An `export` file is owned by the entity that owns the package it represents. The owner of a package defines the API of that package, and may or may not provide all implementations of that package. All implementations, however, must conform to the definition provided in the `export` file provided by the owner.

A particular example of `export` file ownership is the Java Card API packages. Sun defines these packages. Sun also provides the `export` files for these packages. All implementations of the Java Card API packages must conform to the definitions provided by Sun, and comply with the token assignments provided in these `export` files.

Original -

<none>

Section 5.4 Hierarchies Represented

Corrected - Added section to clarify export file format design

Classes and interfaces represented in an `export` file include public elements defined within their respective hierarchies. For example, instead of indicating the immediate superclass or superinterface, all public superclasses or superinterfaces are listed. This design concept is applied not only to superclasses or superinterfaces, but also to virtual methods and implemented interfaces.

Original -

<none>

Section 5.6.3 CONSTANT_Integer

Corrected - Added specification of boolean values

. . .

`bytes`

The `bytes` item of the `CONSTANT_Integer_info` structure contains the value of the `int` constant. The bytes of the value are stored in big-endian (high byte first) order.

The value of a `boolean` type is 1 to represent *true* and 0 to represent *false*.

Original -

. . .

`bytes`

The `bytes` item of the `CONSTANT_Integer_info` structure contains the value of the `int` constant. The bytes of the value are stored in big-endian (high byte first) order.

Section 5.7 Classes and Interfaces

Corrected - Added mention of public flag constraint

. . .

`access_flags`

. . .

All other class access and modifier flags are defined in the same way and with the same restrictions as described in *The Java™ Virtual Machine Specification*.

Since all classes and interfaces represented in an `export` file are public, the `ACC_PUBLIC` flag must always be set.

The Java Card virtual machine reserves all other flag values. Their values must be zero.

. . .

Original -

. . .

`access_flags`

. . .

All other class access and modifier flags are defined in the same way and with the same restrictions as described in *The Java™ Virtual Machine Specification.*

The Java Card virtual machine reserves all other flag values. Their values must be zero.

. . .

Section 5.8 Fields

Corrected - Added mention of public and protected flag constraint

. . .

`access_flags`

. . .

Field access and modifier flags are defined in the same way and with the same restrictions as described in *The Java™ Virtual Machine Specification*.

Since all fields represented in an `export` file are either public or protected, exactly one of the `ACC_PUBLIC` or `ACC_PROTECTED` flag must be set.

The Java Card virtual machine reserves all other flag values. Their values must be zero.

. . .

Original -

. . .

`access_flags`

. . .

Field access and modifier flags are defined in the same way and with the same

restrictions as described in *The Java™ Virtual Machine Specification*.

The Java Card virtual machine reserves all other flag values. Their values must be zero.

. . .

Section 5.8 Fields

Corrected - Added requirement that referenced classes must be public

. . .

`descriptor_index`

. . .

Representation of a field descriptor in an `export` file is the same as in a Java `class` file. See the specification described in *The Java™ Virtual Machine Specification (*§4.3.2).

If this field is a reference-type, the class referenced must be a public class.

Original -

. . .

`descriptor_index`

. . .

Representation of a field descriptor in an `export` file is the same as in a Java `class` file. See the specification described in *The Java™ Virtual Machine Specification (*§4.3.2).

Section 5.9 Methods

Corrected - Added mention of public and protected flag constraint

`access_flags`

. . .

Method access and modifier flags are defined in the same way and with the same restrictions as described in *The Java™ Virtual Machine Specification*.

Since all methods represented in an `export` file are either public or protected, exactly one of the `ACC_PUBLIC` or `ACC_PROTECTED` flag must be set.

Unlike in Java class files, the `ACC_NATIVE` flag is not supported in `export` files. Whether a method is native is an implementation detail that is not relevant to importing packages. The Java Card virtual machine reserves all other flag values. Their values must be zero.

Original -

`access_flags`

. . .

Method access and modifier flags are defined in the same way and with the same restrictions as described in *The Java™ Virtual Machine Specification*.

Unlike in Java class files, the `ACC_NATIVE` flag is not supported in `export` files. Whether a method is native is an implementation detail that is not relevant to importing packages. The Java Card virtual machine reserves all other flag values. Their values must be zero.

Section 5.9 Methods

Corrected - Added requirement that referenced classes must be public

. . .

`descriptor_index`

. . .

Representation of a method descriptor in an `export` file is the same as in a Java class file. See the specification described in *The Java™ Virtual Machine Specification* (§4.3.3).

All classes referenced in a descriptor must be public classes.

Original -

. . .

`descriptor_index`

. . .

Representation of a method descriptor in an `export` file is the same as in a Java class file. See the specification described in *The Java™ Virtual Machine Specification* (§4.3.3).

Section 6.2 Installation

Corrected - Added statement of install order adoption

Installing a `CAP` file onto a Java Card enabled device entails communication between a Java Card enabled terminal and that device. While it is beyond the scope of this specification to define an installation protocol between a terminal and a device, the `CAP` file component order shown in TABLE 6-3 is a reference load order suitable for an implementation with a simple memory management model on a limited memory device.[1]

. . .

1. Both the Java Card Forum and Visa Open Platform specification have adopted this component load order as a standard to enhance interoperability. In both cases the Descriptor Component is not loaded. Furthermore, the Visa Open Platform specification defines the format of packets (APDUs) used during installation.

Original -

Installing a CAP file onto a Java Card enabled device entails communication between a Java Card enabled terminal and that device. While it is beyond the scope of this specification to define an installation protocol between a terminal and a device, the CAP file component order shown in TABLE 6-3 is a reference load order suitable for an implementation with a simple memory management model on a limited memory device.

Section 6.4 Directory Component

Corrected - Removed constraint for AID value

AID[]

The AID item represents the Java Card name of the component. See ISO 7816-5 for the definition of an AID (§4.2).

Each component is assigned an AID conforming to the ISO 7816-5 standard. Beyond that, there are no constraints on the value of an AID of a custom component.

Original -

AID[]

The AID item represents the Java Card name of the component. See ISO 7816-5 for the definition of an AID (§4.2).

Each component is assigned an AID conforming to the ISO 7816-5 standard. The RID (first 5 bytes) of all of the custom component AIDs must have the same value. In addition, the RID of the custom component AIDs must have the same value as the RID of the package defined in this CAP file.

Section 6.7 Constant Pool Component

Corrected - Added description of restriction for first entry

`constant_pool`

. . .

The first entry in the constant pool can not be an exception handler class that is referenced by a `catch_type_index` of an `exception_handler_info` structure. In such a case the value of the `catch_type_index` would be equal to 0, but the value of 0 in a `catch_type_index` is reserved to indicate an `exception_handler_info` structure that describes a *finally* block.

Original -

`constant_pool`

. . .

<none>

Section 6.9 Method Component

Corrected - Added more information to introduction

The Method Component describes each of the methods declared in this package, excluding `<clinit>` methods and interface method declarations. Abstract methods defined by classes (not interfaces) are included. The exception handlers associated with each method are also described.

The Method Component does not contain complete access information and descriptive details for each method. Instead, the information is optimized for size and therefore limited to that required to execute each method without performing verification. Complete details regarding the methods defined in this package are included in the Descriptor Component (§6.13). Among other information, the Descriptor Component contains the location and number of bytecodes for each method in the Method Component. This information can be used to parse the methods in the Method Component.

Instructions and exception handler catch types in the Method Component reference entries in the Constant Pool Component (§6.7). No other CAP file components, including the Method Component, are referenced by the elements in the Method Component.

Original -

The Method Component describes each of the methods declared in this package, excluding <clinit> methods and interface method declarations. The exception handlers associated with each method are also described.

The Method Component does not contain complete access information and descriptive details for each method. Instead, the information is limited to that required to execute each method, without performing verification. Complete details regarding the methods defined in this package are included in the Descriptor Component (§6.13).

Instructions and exception handler catch types in the Method Component reference entries in the Constant Pool Component (§6.7). No other CAP file components, including the Method Component, are referenced by the elements in the Method Component.

Section 6.9 Method Component

Corrected - Revised description of exception handler order

`exception_handlers[]`

The `exception_handlers` item represents an array of 8-byte `exception_handler_info` structures. Each `exception_handler_info` structure represents a `catch` or `finally` block defined in a method of this package.

Entries in the `exception_handlers` array are sorted in ascending order by the the offset to the handler of the exception handler. Smaller offset values occur first in the array. This ordering constraint ensures that the first match found when searching for an exception handler is the correct match.

There are two consequences of this ordering constraint. First, a handler that is nested with the active range (`try` block) of another handler occurs first in the array. Second, when multiple handlers are associated with the same active range, they are ordered as they occur in a method. This is consistent with the ordering constraints defined for Java `class` files. An example is shown below.

```
try {
```

```
        ...
        try {
            ...
        } catch (NullPointerException e) {    // first
            ...
        }
        ...
    } catch (Exception e) {                   // second
        ...
    } finally {                               // third
        ...
    }
    ...
    try {
        ...
    } catch (SecurityException e) {           // fourth
        ...
    }
```

CODE EXAMPLE 4-1 Exception handler example

Original -

`exception_handlers[]`

The `exception_handlers` item represents an array of 8-byte `exception_handler_info` structures. Each `exception_handler_info` structure represents a `catch` or `finally` block defined in a method of this package.

Entries in the `exception_handlers` array are sorted in ascending order by the distance between the beginning of the Method Component to the endpoint of each exception handler range in the `methods` item.

Section 6.9.1 exception_handler_info

Corrected - Added explicit mention of stop_bit

The `exception_handler_info` structure is defined as follows:

```
exception_handler_info {
        u2 start_offset
        u2 bitfield {
                bit[1] stop_bit
                bit[15] active_length
        }
        u2 handler_offset
        u2 catch_type_index
}
```

The items in the `exception_handler_info` structure are as follows:

`start_offset, active_length`

The `start_offset` and `active_length` pair indicate the active range (`try` block) an exception handler. The `start_offset` item indicates the beginning of the active range while the `active_length` item indicates the number of bytes contained in the active range.

end_offset is defined as `start_offset` plus `active_length`.

The `start_offset` item and *end_offset* are byte offsets into the `info` item of the Method Component. The value of the `start_offset` must be a valid offset into a `bytecodes` array of a `method_info` structure to an opcode of an instruction. The value of the *end_offset* either must be a valid offset into a `bytecodes` array of the same `method_info` structure to an opcode of an instruction, or must be equal to the method's bytecode count, the length of the `bytecodes` array of the `method_info` structure. The value of the `start_offset` must be less than the value of the *end_offset*.

The `start_offset` is inclusive and the *end_offset* is exclusive; that is, the exception handler must be active while the execution address is within the interval [`start_offset`, *end_offset*).

`stop_bit`

The `stop_bit` item indicates whether the active range (`try` block) of this exception handler is contained within or is equal to the active range of any succeeding `exception_handler_info` structures in this `exception_handlers` array. At the Java source level, this indicates whether an active range is nested within another, or has at least one succeeding exception handler associated with the same range. The latter occurs when there is at least one succeeding `catch` block or a `finally` block.

The `stop_bit` item is equal to 1 if the active range does not intersect with a succeeding exception handler's active range, and this exception handler is the last handler applicable to the active range. It is equal to 0 if the active range is contained within the active range of another exception handler, or there is at least one succeeding handler applicable to the same active range.

The `stop_bit` provides an optimization to be used during the interpretation of the *athrow* bytecode. As the interpreter searches for an appropriate exception handler, it may terminate the search of the exception handlers in this Method Component under the following conditions:

1. the location of the current program counter is less than the *end_offset* of this exception handler, and

2. the `stop_bit` of this exception handler is equal to 1.

When these conditions are satisfied it is guaranteed that none of the succeeding exception handlers in this Method Component will contain an active range appropriate for the current exception.

In CODE EXAMPLE 4-1 on page 43, the `stop_bit` item is set for both the third and fourth handlers.

`handler_offset`

The `handler_offset` item represents a byte offset into the `info` item of the Method Component. It indicates the start of the exception handler. At the Java source level, this is equivalent to the beginning of a `catch` or `finally` block. The value of the item must be a valid offset into a `bytecodes` array of a `method_info` structure to an opcode of an instruction, and must be less than the value of the method's bytecode count.

`catch_type_index`

If the value of the `catch_type_index` item is non-zero, it must be a valid index into the `constant_pool` array of the Constant Pool Component (§6.7). The `constant_pool` entry at that index must be a `CONSTANT_Classref_info` structure, representing the class of the exception caught by this `exception_handlers` array entry.

If the `exception_handlers` table entry represents a *finally* block, the value of the `catch_type_index` item is zero. In this case the exception handler is called for all exceptions that are thrown within the `start_offset` and *end_offset* range.

The order of constants in the constant pool is constrained such that all entries referenced by `catch_type_index` items that represent `catch` block (not `finally` blocks) are located at non-zero entries.

Original -

The `exception_handler_info` structure is defined as follows:

```
exception_handler_info {
       u2 start_offset
       u2 active_length
       u2 handler_offset
       u2 catch_type_index
}
```

The items in the `exception_handler_info` structure are as follows:

`start_offset, active_length`

The `active_length` item is encoded to indicate whether the active range of

this exception handler is nested within another exception handler. The high bit of the `active_length` item is equal to 1 if the active range is not contained within another exception handler, and this exception handler is the last handler applicable to the active range. The high bit is equal to 0 if the active range is contained within the active range of another exception handler, or there are successive handlers applicable to the same active range.

end_offset is defined as `start_offset` plus `active_length` & 0x7FFF.

The `start_offset` item and *end_offset* are byte offsets into the `info` item of the Method Component. They indicate the ranges in a bytecode array at which the exception handler is active. The value of the `start_offset` must be a valid offset into a `bytecodes` array of a `method_info` structure to an opcode of an instruction. The value of the *end_offset* either must be a valid offset into a `bytecodes` array of a `method_info` structure to an opcode of an instruction or must be equal to a method's bytecode count, the length of the `bytecodes` array of a `method_info` structure. The value of the `start_offset` must be less than the value of the *end_offset*.

The `start_offset` is inclusive and the *end_offset* is exclusive; that is, the exception handler must be active while the execution address is within the interval [`start_offset`, *end_offset*).

`handler_offset`

The handler_offset item represents a byte offset into the `info` item of the Method Component. It indicates the start of the exception handler. The value of the item must be a valid offset into a `bytecodes` array of a `method_info` structure to an opcode of an instruction, and must be less than the value of the method's bytecode count.

`catch_type_index`

If the value of the `catch_type_index` item is non-zero, it must be a valid index into the `constant_pool` array of the Constant Pool Component (§6.7). The `constant_pool` entry at that index must be a `CONSTANT_Classref_info` structure, representing the class of the exception caught by this `exception_handlers` array entry.

If the `exception_handlers` table entry represents a finally block, the value of the `catch_type_index` item is zero. In this case the exception handler is called for all exceptions that are thrown within the `start_offset` and *end_offset* range.

Section 6.10 Static Field Component

Corrected - Added clarity

The Static Field Component contains all of the information required to create and initialize an image of all of the static fields defined in this package, referred to as the *static field image*. Offsets to particular static fields are offsets into the static field image, not the Static Field Component.

`Final static` fields of primitive types are not represented in the static field image. Instead these compile-time constants are placed in line in Java Card instructions.

The Static Field Component includes all information required to initialize classes. In the Java virtual machine a class is initialized by executing its `<clinit>` method. In the Java Card virtual machine the functionality of `<clinit>` methods is represented in the Static Field Component as array initialization data and non-default values of primitive types data. §2.2.4.5 contains a description of the subset of `<clinit>` functionality supported in the Java Card virtual machine.

The Static Field Component does not reference any other component in this `CAP` file. The Constant Pool Component (§6.7), Export Component (§6.12) and Descriptor Component (§6.13) reference fields in the static field image defined by the Static Field Component.

The ordering constraints, or segments, associated with a static field image are shown in TABLE 4-1. `Reference` types occur first in the image. Arrays initialized through Java `<clinit>` methods occur first within the set of `reference` types. Primitive types occur last in the image, and primitive types initialized to non-default values occur last within the set of primitive types.

category	segment	content
`reference` types	1	arrays of primitive types initialized by `<clinit>` methods
	2	`reference` types initialized to `null`, including arrays
primitive types	3	primitive types initialized to default values
	4	primitive types initialized to non-default values

TABLE 4-1 Segments of a static field image

The result of an arithmetic expression produced by a Java Card virtual machine must be equal to the result produce by a Java virtual machine, regardless of the input values. A Java Card virtual machine that does not support the `int` data type must reject expressions that could produce a different result.

Original -

The Static Field Component contains all of the information required to create and initialize an image of all of the static fields defined in this package, referred to as the *static field image*. `Final static` fields of primitive types are not represented in the static field image. Instead these compile-time constants are placed in line in Java Card instructions.

The Static Field Component does not reference any other component in this `CAP` file. The Constant Pool Component (§6.7), Export Component (§6.12) and Descriptor Component (§6.13) reference fields defined in the Static Field Component.

The ordering constraints, or segments, associated with a static field image are shown in TABLE 4-1. `Reference` types occur first in the image. Arrays initialized through Java `<clinit>` methods occur first within the set of `reference` types. Primitive types occur last in the image, and primitive types initialized to non-default values occur last within the set of primitive types.

category	segment	content
`reference` types	1	arrays of primitive types initialized by `<clinit>` methods
	2	`reference` types initialized to `null`
primitive types	3	primitive types initialized to default values
	4	primitive types initialized to non-default values

TABLE 4-2 Segments of a static field image

Section 6.10 Static Field Component

Corrected - Added image size formula

`image_size`

...

The value of the `image_size` is defined as:

```
image_size =
       reference_count * 2 +
       default_value_count  +
       non_default_value_count.
```

Original -

`image_size`

...

<none>

Section 6.10 Static Field Component

Corrected - Added definition of boolean values

`non_default_values[]`

The `non_default_values` item represents an array of bytes of non-default initial values. This is the exact image of segment 4 of the static field image as described in TABLE 4-1. The number of entries in the `non_default_values` array for each static field type is equal to the size in bytes of the type as shown in TABLE 6-9.

The value of a `boolean` type is 1 to represent *true* and 0 to represent *false*.

Original -

The `int` keyword and 32-bit integer data types need not be supported in a Java Card implementation. A Java Card virtual machine that does not support the `int` data type will reject programs which use the `int` data type or 32-bit intermediate values.

Section 6.11 Reference Location Component

Corrected - Added explicit mention of exception handlers

The Reference Location Component represents lists of offsets into the `info` item of the Method Component (§6.9) to items that contain indices into the `constant_pool` array of the Constant Pool Component (§6.7). This includes all constant pool index operands of instructions, and all non-zero `catch_type_index` items of the `exception_handlers` array. The `catch_type_index` items that have the value of 0 are not included since they represent `finally` blocks instead of particular exception classes.

Some of the constant pool indices are represented in one-byte values while others are represented in two-byte values. Operands of *getfield_T* and *putfield_T* instructions are one-byte constant pool indices. All other indices in a Method Component are two-byte values.

The Reference Location Component is not referenced by any other component in this `CAP` file.

. . .

Original -

The Reference Location Component represents lists of offsets into the `info` item of the Method Component (§6.9) to operands that contain indices into the `constant_pool` array of the Constant Pool Component (§6.7). Some of the constant pool indices are represented in one-byte values while others are represented in two-byte values.

The Reference Location Component is not referenced by any other component in this `CAP` file.

. . .

Section 6.13.1 class_descriptor_info

Corrected - Added statements regarding static final fields

`field_count`

The `field_count` item represents the number of entries in the `fields` array. If this `class_descriptor_info` structure represents an interface, the value of the `field_count` item is equal to zero.

`Static final` fields of primitive types are not represented as fields in a CAP file, but instead these compile-time constants are placed inline in bytecode sequences. The `field_count` item does not include static final field of primitive types defined by this class.

`...`

`fields[]`

The `fields` item represents an array of `field_descriptor_info` structures. Each field declared by this class is represented in the array, except `static final` fields of primitive types.

Original -

`field_count`

The `field_count` item represents the number of entries in the `fields` array. If this `class_descriptor_info` structure represents an interface, the value of the `field_count` item is equal to zero.

`...`

`fields[]`

The `fields` item represents an array of `field_descriptor_info` structures. Each field declared by this class is represented in the array.

Section 6.13.2 field_descriptor_info

Corrected - Definition of instance field reference

```
field_descriptor_info {
       u1 token
       u1 access_flags
       union {
              static_field_ref static_field
              {
                class_ref class
                u1 token
              } instance_field
       } field_ref
       union {
              u2 primitive_type
              u2 reference_type
       } type
}
```

`...`

`field_ref`

The `field_ref` item represents a reference to this field. If the `ACC_STATIC` flag is equal to 1, this item represents a `static_field_ref` as defined in the `CONSTANT_StaticFieldref` structure (§6.7.3).

If the `ACC_STATIC` flag is equal to 0, this item represents a reference to an

instance field. It contains a `class_ref` item and an instance field `token` item. These items are defined in the same manner as in the `CONSTANT_InstanceFieldref` structure (§6.7.2).

Original -

```
field_descriptor_info {
       u1 token
       u1 access_flags
       union {
              static_field_ref static_field
              instance_field_ref instance_field
       } field_ref
       union {
              u2 primitive_type
              u2 reference_type
       } type
}
```

...

`field_ref`

The `field_ref` item represents a reference to this field. If the `ACC_STATIC` flag is equal to 1, this item represents a `static_field_ref` as defined in the `CONSTANT_StaticFieldref` structure (§6.7.3).

If the `ACC_STATIC` flag is equal to 0, this item represents an `instance_field_ref` as defined in the `CONSTANT_InstanceFieldref` structure (§6.7.2).

Section 6.13.3 method_descriptor_info

Corrected - Added information regarding parsing methods

The `method_descriptor_info` structure is used to describe a method defined in this package. This structure contains sufficient information to locate and parse the methods in the Method Component, while the Method Component does not.

Original -

The `method_descriptor_info` structure is used to describe a method defined in this package:

Chapter 7 Instruction Set, aastore

Corrected - Added “at runtime” qualification

. . .

At runtime the type of value must be confirmed to be assignment compatible with the type of the components of the array referenced by *arrayref*. Assignment of a value of reference type *S* (source) to a variable of reference type *T* (target) is allowed only when the type *S* supports all of the operations defined on type *T*. The detailed rules follow:

. . .

Original -

. . .

The type of value must be assignment compatible with the type of the components of the array referenced by arrayref. Assignment of a value of reference type *S* (source) to a variable of reference type *T* (target) is allowed only when the type *S* supports all of the operations defined on type *T*. The detailed rules follow:

. . .

Chapter 7 Instruction Set, aastore, checkcast, interfaceof

Corrected - Added description of occurrences of array of interface types, corrected multi-dimensional array footnote

. . .

- If *S* is an interface type[1], then:
  - If *T* is a class type, then *T* must be `Object` (§2.2.2.4);
  - If *T* is an interface type, *T* must be the same interface as *S* or a superinterface of *S*.
- If *S* is an array type, namely the type *SC*`[]`, that is, an array of components of type *SC*, then:
  - If *T* is a class type, then *T* must be `Object`.

- If *T* is an array type, namely the type *TC*`[]`, an array of components of type *TC*, then one of the following must be true:
  - *TC* and *SC* are the same primitive type (§3.1).
  - *TC* and *SC* are reference types[2] (§3.1) with type *SC* assignable to *TC*, by these rules.

. . .

---

1. When both S and T are arrays of reference types, this algorithm is applied recursively using the types of the arrays, namely SC and TC. In the recursive call, S, which was SC in the original call, may be an interface type. This rule can only be reached in this manner. Similarly, in the recursive call, T, which was TC in the original call, may be an interface type.
2. This version of the Java Card virtual machine specification does not support multi-dimensional arrays. Therefore, neither SC or TC can be an array type.

Original -

. . .

- If *S* is an interface type, then:
  - If *T* is a class type, then *T* must be `Object` (§2.2.2.4);
  - If *T* is an interface type, *T* must be the same interface as *S* or a superinterface of *S*.
- If *S* is an array type[1], namely the type *SC*`[]`, that is, an array of components of type *SC*, then:
  - If *T* is a class type, then *T* must be `Object`.
  - If *T* is an array type, namely the type *TC*`[]`, an array of components of type *TC*, then one of the following must be true:
    - *TC* and *SC* are the same primitive type (§3.1).
    - *TC* and *SC* are reference types (§3.1) with type *SC* assignable to *TC*, by these rules.

. . .

---

1. This version of the Java Card virtual machine specification does not allow for arrays of more than one dimension. Therefore, neither S or T can be an array type, and the rules for array types do not apply.

Chapter 7 Instruction Set, getfield_<t>, getfield_<t>_this, getfield_<t>_w, putfield_<t>, putfield_<t>_this, putfield_<t>_w

Corrected - Corrected qualification of access to protected fields

. . .

The currently executing method must be an instance method. The local variable at index 0 must contain a `reference` *objectref* to the currently executing method's `this` parameter. The unsigned *index* is used as an index into the constant pool of the current package (§3.5). The constant pool item at the index must be of type `CONSTANT_InstanceFieldref` (§6.7.2), a reference to a class and a field token.

The class of *objectref* must not be an array. If the field is `protected`, and it is a member of a superclass of the current class, and the field is not declared in the same package as the current class, then the class of *objectref* must be either the current class or a subclass of the current class.

. . .

Original -

. . .

The *objectref*, which must be of type `reference`, is popped from the operand stack. The unsigned *index* is used as an index into the constant pool of the current package (§3.5). The constant pool item at the index must be of type `CONSTANT_InstanceFieldref` (§6.7.2), a reference to a class and a field token. If the field is `protected`, then it must be either a member of the current class or a member of a superclass of the current class, and the class of *objectref* must be either the current class or a subclass of the current class.

. . .

Chapter 7 Instruction Set, getfield_<t>, getfield_<t>_this, getfield_<t>_w, putfield_<t>, putfield_<t>_this, putfield_<t>_w

Corrected - Added footnote describing computation of offset

. . .

The width of a field in a class instance is determined by the field type specified in the instruction. The item is resolved, determining the field offset[1]. The *value* at that offset into the class instance referenced by *objectref* is fetched. If the *value* is of type `byte` or type `boolean`, it is sign-extended to a short. The *value* is pushed onto the operand stack.

. . .

---

1. The offset may be computed by adding the field token value to the size of an instance of the immediate superclass. However, this method is not required by this specification. A Java Card virtual machine may define any mapping from token value to offset into an instance.

Original -

. . .

The width of a field in a class instance is determined by the field type specified in the instruction. The item is resolved, determining the field offset. The *value* at that offset into the class instance referenced by *objectref* is fetched. If the *value* is of type `byte` or type `boolean`, it is sign-extended to a short. The *value* is pushed onto the operand stack.

. . .

Chapter 7 Instruction Set, invokespecial

Corrected - Corrected qualification of access to protected methods

. . .

The unsigned *indexbyte1* and *indexbyte2* are used to construct an *index* into the constant pool of the current package (§3.5), where the value of the index is (*indexbyte1* << 8) | *indexbyte2*. If the invoked method is a private instance method or an instance initialization method, the constant pool item at *index* must be of type `CONSTANT_StaticMethodref` (§6.7.3), a reference to a statically linked instance method. If the invoked method is a superclass method, the constant pool item at *index* must be of type `CONSTANT_SuperMethodref` (§6.7.2), a reference to an instance method of a specified class. The reference is resolved. The resolved method must not be `<clinit>`, a class or interface initialization method. If the method is `<init>`, an instance initialization method, then the method must only be invoked once on an uninitialized object, and before the first backward branch following the execution of the *new* instruction that allocated the object. Finally, if the resolved method is `protected`, and it is a member of a superclass of the current class, and the method is not declared in the same package as the current class, then the class of *objectref* must be either the current class or a subclass of the current class.

. . .

Original -

. . .

The unsigned *indexbyte1* and *indexbyte2* are used to construct an *index* into the constant pool of the current package (§3.5), where the value of the index is (*indexbyte1* << 8) | *indexbyte2*. If the invoked method is a private instance method or an instance initialization method, the constant pool item at *index* must be of type `CONSTANT_StaticMethodref` (§6.7.3), a reference to a statically linked instance method. If the invoked method is a superclass method, the constant pool item at *index* must be of type `CONSTANT_SuperMethodref` (§6.7.2), a reference to an instance method of a specified class. The reference is resolved. The resolved method must not be `<clinit>`, a class or interface initialization method. If the method is `<init>`, an instance initialization method, then the method must only be invoked once on an uninitialized object, and before the first backward branch following the execution of the *new* instruction that allocated the object. Finally, if the method is `protected`, then it must be either a member of the current class or a member of a superclass of the current class, and the class of *objectref* must be either the current class or a subclass of the current class.

. . .

Chapter 7 Instruction Set, invokestatic

Corrected - Removed qualification of access to protected static methods

. . .

The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the constant pool of the current package (§3.5), where the value of the index is (*indexbyte1* << 8) | *indexbyte2*. The constant pool item at that index must be of type `CONSTANT_StaticMethodref` (§6.7.3), a reference to a static method. The method must not be `<init>`, an instance initialization method, or `<clinit>`, a class or interface initialization method. It must be `static`, and therefore cannot be `abstract`.

. . .

Original -

. . .

The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the constant pool of the current package (§3.5), where the value of the index is (*indexbyte1* << 8) | *indexbyte2*. The constant pool item at that index must be of type `CONSTANT_StaticMethodref` (§6.7.3), a reference to a static method. The method must not be `<init>`, an instance initialization method, or `<clinit>`, a class or interface initialization method. It must be `static`, and therefore cannot be `abstract`. Finally, if the method is `protected`, then it must be either a member of the current class or a member of a superclass of the current class.

. . .

Chapter 7 Instruction Set, invokevirtual

Corrected - Corrected qualification of access to protected methods

. . .

The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the constant pool of the current package (§3.5), where the value of the index is (*indexbyte1* << 8) | *indexbyte2*. The constant pool item at that index must be of type `CONSTANT_VirtualMethodref` (§6.7.2), a reference to a class and a virtual method token. The specified method is resolved. The method must not be `<init>`, an instance initialization method, or `<clinit>`, a class or interface initialization method. Finally, if the resolved method is `protected`, and it is a member of a superclass of the current class, and the method is not declared in the same package as the current class, then the class of *objectref* must be either the current class or a subclass of the current class.

. . .

Original -

. . .

The unsigned *indexbyte1* and *indexbyte2* are used to construct an index into the constant pool of the current package (§3.5), where the value of the index is (*indexbyte1* << 8) | *indexbyte2*. The constant pool item at that index must be of type `CONSTANT_VirtualMethodref` (§6.7.2), a reference to a class and a virtual method token. The specified method is resolved. The method must not be `<init>`, an instance initialization method, or `<clinit>`, a class or interface initialization method. If the method is `protected`, then it must be either a member of the current class or a member of a superclass of the current class, and the class of *objectref* must be either the current class or a subclass of the current class.

. . .

Chapter 7 Instruction Set, sshl, sshr

Corrected - Added justification for shift mask value

This is equivalent (even if overflow occurs) to multiplication by 2 to the power *s*. The shift distance actually used is always in the range 0 to 31, inclusive, as if *value2* were subjected to a bitwise logical AND with the mask value 0x1f.

The mask value of 0x1f allows shifting beyond the range of a 16-bit `short` value. It is used by this instruction, however, to ensure results equal to those generated by the Java instruction `ishl`/`ishr`.

Original -

This is equivalent (even if overflow occurs) to multiplication by 2 to the power *s*. The shift distance actually used is always in the range 0 to 31, inclusive, as if *value2* were subjected to a bitwise logical AND with the mask value 0x1f.

Chapter 7 Instruction Set, sushr

Corrected - Added description of equivalence with Java and justification for shift mask value

Description

Both *value1* and *value2* must be of type `short`. The values are popped from the operand stack. A `short` *result* is calculated by sign-extending *value1* to 32 bits[1] and shifting the result right by *s* bit positions, with zero extension, where *s* is the value of the low five bits of *value2*. The resulting value is then truncated to a 16-bit *result*. The *result* is pushed onto the operand stack.

Notes

If *value1* is positive and *s* is *value2* & 0x1f, the result is the same as that of *value1* >> *s*; if *value1* is negative, the result is equal to the value of the expression (*value1* >> *s*) + (2 << ~*s*). The addition of the (2 << ~*s*) term cancels out the propagated sign bit. The shift distance actually used is always in the range 0 to 31, inclusive, as if *value2* were subjected to a bitwise logical AND with the mask value 0x1f.

The mask value of 0x1f allows shifting beyond the range of a 16-bit `short` value. It is used by this instruction, however, to ensure results equal to those generated by the Java instruction `iushr`.

---

1. Sign extension to 32 bits ensures that the result computed by this instruction will be exactly equal to that computed by the Java `iushr` instruction, regardless of the input values. In a Java Card virtual machine the expression "0xffff `>>>` 0x01" yields 0xffff, where ">>>" is performed by the `sushr` instruction. The same result is rendered by a Java virtual machine.

Original -

Description

Both *value1* and *value2* must be of type `short`. The values are popped from the operand stack. A `short` *result* is calculated by sign-extending *value1* to 32 bits and shifting the result right by *s* bit positions, with zero extension, where *s* is the value of the low five bits of *value2*. The resulting value is then truncated to a 16-bit *result*. The *result* is pushed onto the operand stack.

Notes

If *value1* is positive and *s* is *value2* & 0x1f, the result is the same as that of *value1* >> *s*; if *value1* is negative, the result is equal to the value of the expression (*value1* >> *s*) + (2 << ~*s*). The addition of the (2 << ~*s*) term cancels out the propagated sign bit. The shift distance actually used is always in the range 0 to 31, inclusive, as if *value2* were subjected to a bitwise logical AND with the mask value 0x1f.